

# Standard Operating Procedure

## PYTHON SCRIPT COMPILATION TO STANDALONE EXECUTABLES

|                   |                                                   |                     |                    |
|-------------------|---------------------------------------------------|---------------------|--------------------|
| Project Context:  | LAB-IQ / HFRR Advanced (WO 2472)                  | Target Environment: | Windows OS (x64)   |
| Source Directory: | C:\LAB-IQ\HFRR Adv\WO 2472\src\                   | Compiler Engine:    | PyInstaller Engine |
| Target Scripts:   | 1. advanced_roi_editor11.py<br>2. livestream10.py |                     |                    |

### 1. Purpose & Scope

This document details the deployment process for converting the development Python modules of the **HFRR Advanced** suite into production-ready, standalone Windows Executables ( `.exe` ). This process ensures that the modules run seamlessly on client infrastructure without requiring a local installation of Python, specific environment configurations, or raw source-code access.

### 2. System Prerequisites

Before starting the compilation pipeline, verify that the active Windows Environment fulfills the following structural baselines:

- **Python Core:** Python 3.8 to 3.12 (64-bit architecture recommended).
- **Environment Path:** System `PATH` environmental variables must correctly map to your Python deployment and its corresponding `\Scripts` directory.
- **Permissions:** An administrative or high-privilege Command Prompt instance to ensure unobstructed local folder read/write cycles.

### 3. Step-by-Step Compilation Protocol

#### STEP 1

#### Environment Optimization & Tool Installation

Open a Command Prompt window and execute the following dependency mapping. This guarantees that PyInstaller is updated to compile standard assets securely.

```
C:\> pip install --upgrade pyinstaller
```

#### STEP 2

#### Directory Focus Positioning

Switch your terminal context directly to the active workspace folder. Wrapping paths containing spaces in double quotes is critical to prevent string parsing faults.

```
C:\> cd "C:\LAB-IQ\HFRR Adv\W0 2472\src"
```

### STEP 3

## Execution of Compilation Strings

Determine the execution context of your application to select the appropriate terminal syntax variant below.

### Variant A: GUI Application Deployment (Recommended)

Use these configurations if the application uses an interactive Graphical User Interface (e.g., PyQt, Tkinter) and you want to prevent a blank command terminal from showing behind the UI.

```
# Build Command for ROI Editor App
C:\LAB-IQ\HFRR Adv\W0 2472\src> pyinstaller --onefile --windowed
advanced_roi_editor11.py

# Build Command for Livestream App
C:\LAB-IQ\HFRR Adv\W0 2472\src> pyinstaller --onefile --windowed livestream10.py
```

### Variant B: Console/Terminal-Dependent App Deployment

Use these configurations if your applications depend entirely on standard print commands, data streams, or log outputs to a visible console window.

```
# Build Command for ROI Editor App (Console Visible)
C:\LAB-IQ\HFRR Adv\W0 2472\src> pyinstaller --onefile advanced_roi_editor11.py

# Build Command for Livestream App (Console Visible)
C:\LAB-IQ\HFRR Adv\W0 2472\src> pyinstaller --onefile livestream10.py
```

## 4. Parameter Architecture Dictionary

Understanding flags utilized during generation maintains standardization across iterative development sprints:

- **--onefile** : Packages your script into a single, standalone executable payload. It self-extracts into runtime temporary folders dynamically during boot.
- **--windowed** (or **-w**) : Suppresses standard Windows consoles. Essential for high-polish client-facing graphic application tools.

## 5. Output Architecture & Workspace Management

After the build finishes successfully, PyInstaller builds out three structural items inside your source folder:

1. **dist/ (Distribution Directory):** This is your main folder. It contains your finalized standalone executable programs:
  - `..\src\dist dvanced_roi_editor11.exe`
  - `..\src\dist\livestream10.exe`
2. **build/ Directory:** A workspace directory where PyInstaller maps out logs, caches, and tracking details. This can be safely discarded after a complete build run.
3. **\*.spec Files:** Configuration scripts for your build settings. Keep these if you plan to run automated, complex rebuild pipelines later.

### Post-Build Cleanup Task

Once you verify that your `.exe` files in the `dist/` folder launch correctly, you can delete the `build/` folder and all `.spec` files to clean up your workspace.

### Important: Asset Path Handling

If your scripts load external files like images, configurations, or hardware drivers using relative paths (e.g., `open("config.json")`), copy those assets into the new `dist/` directory alongside your `.exe` files so they can be found at runtime.